WEEK 2 · DAY 4 · OVERVIEW

Duration and DV01



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

We hold 5,000,000 Quorvane, 3,000,000 Dunmarrow, 4,000,000 Pellwick and 2,000,000 Halvering, settling Monday 30 November. If yields move one basis point, what does each position make or lose in dollars, and the book? Give bondmath duration and DV01, tied to Excel. Last, September's holdings file has a duration column that does not say which duration it is: find out.

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

Your Route Through Day 4

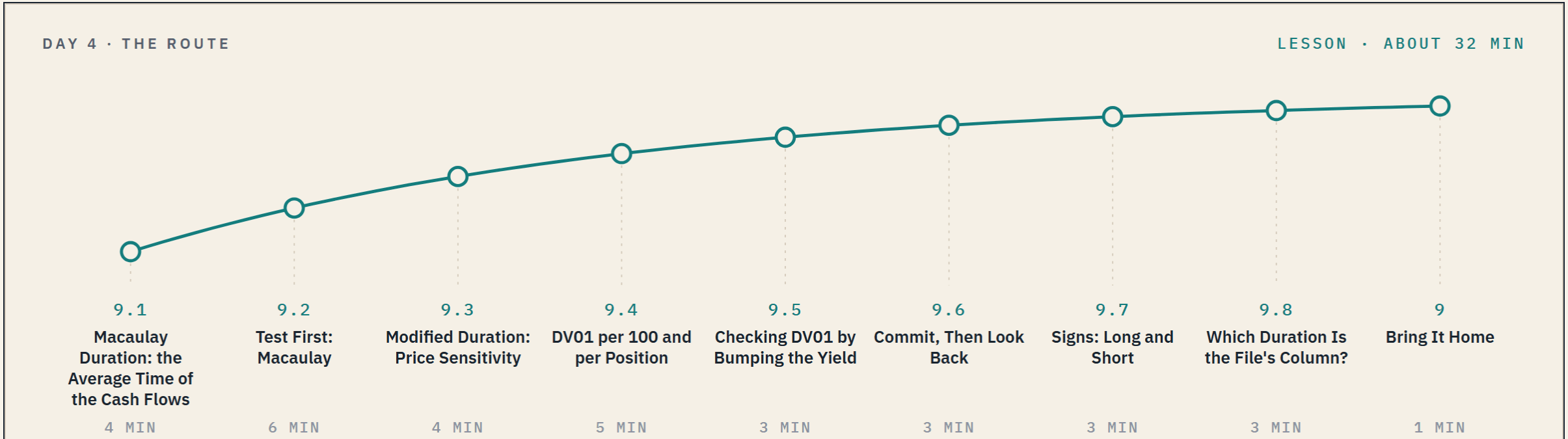


FIGURE 1 Nine sections, in order, with about how long each takes.

Time. About 32 minutes for the lesson, then about 12 for the four exercises.

9.1 Macaulay Duration: the Average Time of the Cash Flows

EACH PAYMENT'S SHARE OF THE PRICE

	TIME_YEARS	PV	WEIGHT
0	0.333333	2.578788	0.025727
1	0.833333	2.510991	0.025051
2	1.333333	2.444977	0.024392
3	1.833333	2.380698	0.023751
4	2.333333	2.318109	0.023126
5	2.833333	2.257166	0.022518
6	3.333333	2.197825	0.021926
7	3.833333	2.140043	0.021350
8	4.333333	2.083781	0.020789
9	4.833333	79.324172	0.791370

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Times, cash flows, present values, B2 to D2 filled down	`=(120/180+A2)/2`, `=IF(A2=9,5.25/2+100,5.25/2)`, `=C2/(1+5.4/100/2)^(120/180+A2)`	0.333333 to 4.833333 years

9.2 Test First: macaulay_duration

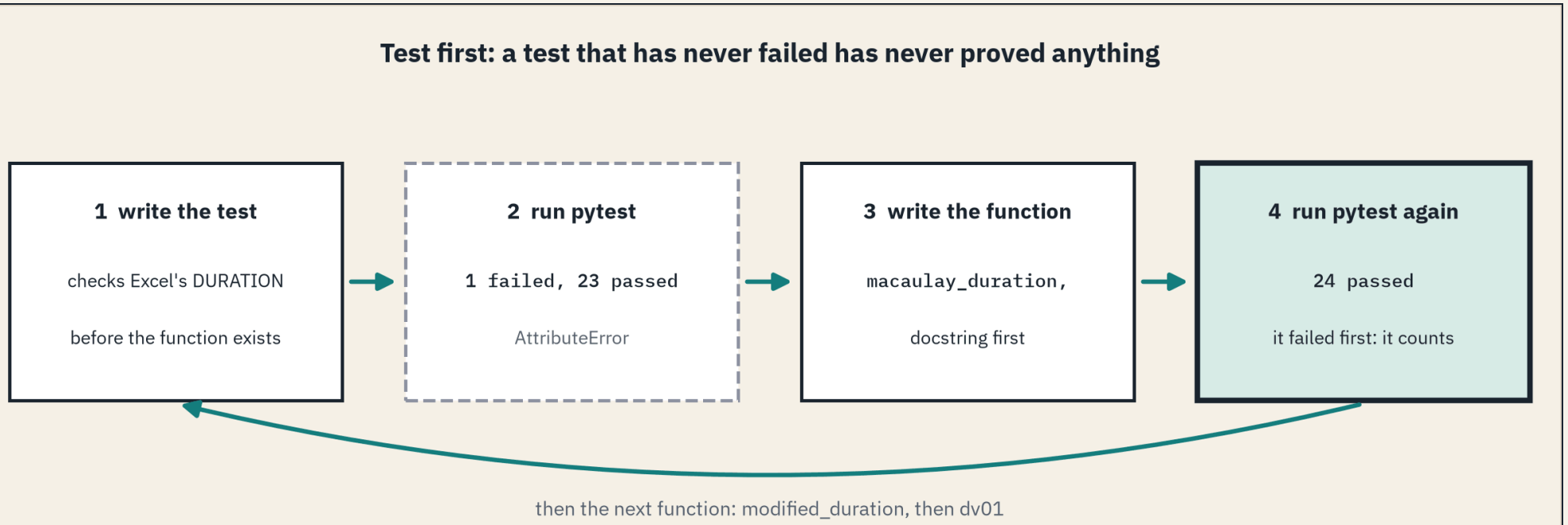


FIGURE 3 Test first: write the test, watch it fail, write the function, and run it again to 24 passed.

> `macaulay_duration` **is Excel's** `DURATION` **on all 306 rows, and its test failed first, so it proved something.**

Test First, Then the Function

THE TEST, WRITTEN FIRST

```
%%writefile -a test_bondmath.py
def test_macaulay_matches_excel_duration():
    for row in REFERENCE:
        result = bondmath.macaulay_duration(*bond_inputs(row))
        assert result == pytest.approx(float(row["duration"]), abs=1e-6), row["case_id"]
```

THE FUNCTION

```
    for k in range(coupons_left):
        cash_flow = coupon + 100 if k == coupons_left - 1 else coupon
        time_years = (w + k) / frequency
        pv = cash_flow / (1 + period_yield) ** (w + k)
    return weighted_time / total_pv
```

9.3 Modified Duration: Price Sensitivity

ONE BASIS POINT, REPRICED

```
dirty_now = bondmath.dirty_price(settlement, quorvane_maturity, 5.25, 5.40)
dirty_up = bondmath.dirty_price(settlement, quorvane_maturity, 5.25, 5.41)
print(f'Repriced:          {(dirty_up / dirty_now - 1) * 100:.8f} percent')
print(f'Modified duration: {-modified_by_hand * 0.0001 * 100:.8f} percent')
```

```
Repriced:          -0.04179333 percent
Modified duration: -0.04180378 percent
```

FOUR BONDS AGAINST MDURATION

	ISSUER	MODIFIED	EXCEL
0	Quorvane Energy	4.180378	4.180378
1	Dunmarrow Rail	5.462335	5.462335
2	Pellwick Paper	2.605127	2.605127
3	Halvering Foods	6.759766	6.759766

Quorvane's price moves about 0.0418 percent for each basis point.

Modified Duration in Excel

DIVIDE BY ONE PLUS THE PERIOD YIELD

```
modified_by_hand = macaulay_by_hand / (1 + 5.40 / 100 / 2)
print(f'Modified duration: {modified_by_hand:.6f}')
```

```
Modified duration: 4.180378
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Modified duration	`=MDURATION(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,2,0)`	4.180377687066382
The division	`=DURATION(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,2,0)/(1+5.4/100/2)`	4.180377687066382

9.4 DV01 per 100 and per Position

DV01 PER 100

	ISSUER	DV01	EXCEL
0	Quorvane Energy	0.041903	0.041903
1	Dunmarrow Rail	0.055175	0.055175
2	Pellwick Paper	0.026359	0.026359
3	Halvering Foods	0.067511	0.067511

PRICE POINTS PER BP

```
bonds['dv01'] = [bondmath.dv01(settlement, m, c, y)
bonds['excel'] = week2.loc[bonds['cusip'], 'dv01'].to_numpy()
assert (bonds['dv01'] - bonds['excel']).abs().max() < 1e-8
bonds[['issuer', 'par_held', 'dv01', 'excel']].round(6)
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Modified duration, B4	`=MDURATION(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,2,0)`	4.180377687066382
DV01 per 100, B5	`=B4*(B1+B2)/10000`	0.04190266430092781

9.5 Checking DV01 by Bumping the Yield

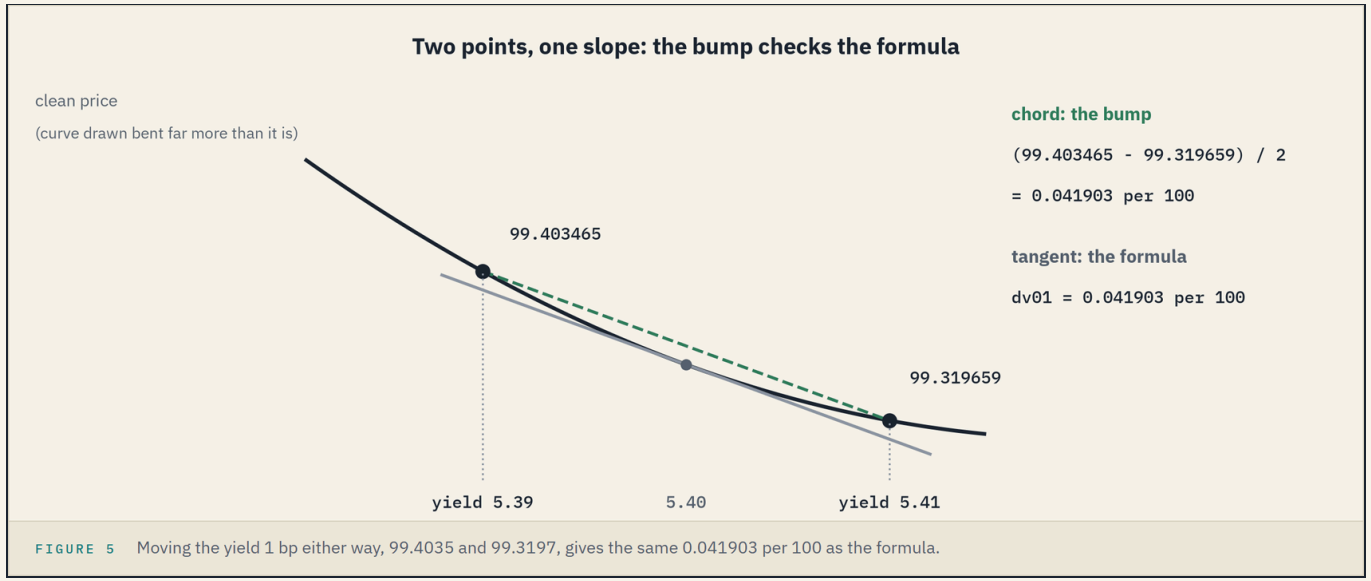


FIGURE 5 Moving the yield 1 bp either way, 99.4035 and 99.3197, gives the same 0.041903 per 100 as the formula.

PRICE IT 1 BP EITHER SIDE

	ISSUER	DV01	BUMP
0	Quorvane Energy	0.041903	0.041903
1	Dunmarrow Rail	0.055175	0.055175
2	Pellwick Paper	0.026359	0.026359
3	Halvering Foods	0.067511	0.067511

Two routes, one number: the formula and the bump agree to 1e-08 per 100.

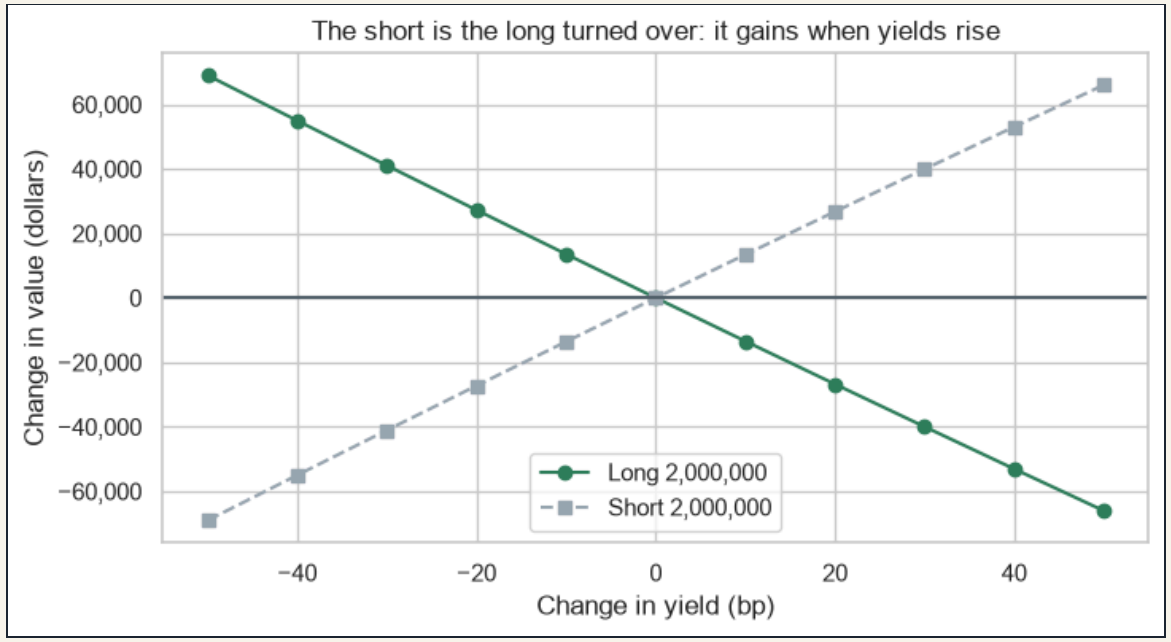
9.6 Commit, Then Look Back

READ THE DIFF

```
!git -C "{work}" diff --stat
```

```
 bondmath.py      | 55 ++++++
```

9.7 Signs: Long and Short



THE BOOK, 1 BP EITHER WAY

```
1 bp: estimate -6,154.97, repriced -6,153.13 dollars
-1 bp: estimate 6,154.97, repriced 6,156.81 dollars
```

A SHORT POSITION

```
Short DV01: -1,350.23 dollars per basis point
Change for a 1 bp rise: 1,350.23 dollars
```

A long gains its DV01 when yields fall 1 bp; a short carries a negative DV01 and gains when they rise.

9.8 Which Duration Is the File's Column?

BOTH DURATIONS, ALL 200 BONDS

```
holdings['macaulay_python'] = [bondmath.macaulay_duration(*bond) for bond in inputs]
holdings['modified_python'] = [bondmath.modified_duration(*bond) for bond in inputs]
```

WITHIN ROUNDING ON EVERY BOND

```
holdings['gap'] = (holdings['modified_python'] - holdings['duration']).abs()
assert holdings['gap'].max() <= 0.0005 + 1e-9
same = (holdings['modified_python'].round(3) == holdings['duration']).sum()
print(same, 'of', len(holdings), 'bonds round to the file value')
```

```
200 of 200 bonds round to the file value
```

The file's `duration` column is modified duration, on all 200 bonds.

Bring It Home

COPY YOUR MODULE HOME AND COMMIT

```
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$env:TEMP\pfi_course2_09_k4w7q2xr\bondmath.py", "$env:TEMP\pfi_course2_09_k4w7q2xr\test_bondmath.py" .
python -m pytest -q
git diff --stat
git commit -am "Add macaulay_duration, modified_duration, and dv01, tested against Excel"
git log --oneline -- bondmath.py
git diff HEAD~1 -- bondmath.py
```

pytest shows 27 passed, and the log lists every day that changed your module.

You Can Now

- Compute Macaulay duration by hand, as a value-weighted average time, and in a sheet with **SUMPRODUCT**.
- Write a test first and watch it fail before the function exists.
- Compute Macaulay and modified duration with **bondmath**, tied to **DURATION** and **MDURATION**.
- Turn modified duration into DV01 per 100, per position, and for the book, in dollars.
- Check DV01 by bumping the yield 1 bp either way.
- Read the sign of DV01 for a long and a short position.
- Ask Git which commits changed one file, and what the last one added.

NEXT
WEEK 2 · DAY 5

Case Study: Convexity and the September Book

The bend duration's straight line leaves out, measured and tied to Excel, then a book of 200 bonds rebuilt with the week's module and added up in dollars per basis point.